1. Project definition

Project name: ThreadForge
Project type: High-performance task scheduling runtime
Language: C++20
Primary goal: Execute many tasks efficiently across CPU cores using multiple scheduling strategies, especially **work stealing**.

The simplest way to think about it:

```
Plain text
User creates tasks
        ↓
ThreadForge scheduler
        ↓
Worker threads
        ↓
Each worker owns a queue
        ↓
Idle workers steal work from busy workers
        ↓
Tasks execute in parallel
        ↓
Metrics are collected
```

ThreadForge should eventually be something like a miniature version of the scheduling layer found inside game engines, rendering systems, compilers, scientific-computing frameworks, and parallel runtimes.

---

2. What problem are we solving?

2. What problem are we solving?

Suppose a program has:

```
Plain text
100,000 independent jobs
```

and the machine has:

```
Plain text
8 CPU cores
```

Executing the jobs sequentially wastes available CPU resources.

A naive solution might create one thread per task.

That is also bad because:

```
Plain text
100,000 tasks
→ 100,000 threads
→ huge scheduling overhead
→ excessive memory usage
→ excessive context switching
```

Instead, ThreadForge creates approximately:

```
Plain text
N CPU cores
→ N worker threads
```

and distributes tasks among those workers.

The hard part is deciding:

The hard part is deciding:

- which worker executes which task,
- what happens when one worker finishes early,
- how task priorities are handled,
- how dependent tasks are handled,
- how synchronization is performed,
- how contention is reduced,
- how shutdown and cancellation work.

That is the actual engineering challenge.

---

3. Core requirements

The first production-quality version should contain these major components.

Component	Requirement
Thread Pool	Fixed configurable number of worker threads
Task API	Submit arbitrary callable C++ tasks
Futures	Retrieve return values from tasks
Local Queues	Each worker maintains its own task queue
Work Stealing	Idle workers steal tasks from other workers
Global Queue	Handle externally submitted tasks
Priorities	High / Normal / Low priority
Dependencies	Tasks may depend on other tasks
Synchronization	Atomics, mutexes, condition variables
Shutdown	Graceful and immediate shutdown

Shutdown	Graceful and immediate shutdown
Metrics	Track scheduler performance
Benchmarks	Compare different scheduling strategies
Tests	Unit, stress, race-condition, and correctness tests

---

4. Functional requirements

Task submission

The user should eventually be able to write something similar to:

```cpp
ThreadForge scheduler(8);

auto result = scheduler.submit([] {
    return expensive_computation();
});

std::cout << result.get();
```

Tasks should support:

```cpp
void functions
functions returning values
lambdas
function objects
functions with arguments
```

For example:

```cpp
```

```cpp
auto future = scheduler.submit([](int a, int b) {
    return a + b;
}, 10, 20);
```

---

5. Worker architecture

Suppose there are four worker threads:

```
Worker 0
Queue: [A B C D]

Worker 1
Queue: [E F]

Worker 2
Queue: []

Worker 3
Queue: [G H I]
```

Worker 2 is idle.

Instead of sleeping immediately, it should steal work:

```
Worker 2
   ↓
looks at Worker 0
   ↓
steals task A
   ↓
executes A
```

Now CPU resources stay utilized.

This is the central feature of ThreadForge.

---

6. Work-stealing design

A useful strategy is:

```
Plain text
Owner worker:
push_front()
pop_front()

Stealing worker:
pop_back()
```

Why?

Because the owner works mostly with one end of the queue while thieves access the other end, reducing contention.

Example:

```
Plain text
Worker 1 Queue

FRONT                    BACK

Task E
Task D
Task C
Task B
Task A

Worker 1 takes E
Worker 2 steals A
```

Initially the queue can be mutex protected.

Later you can attempt a more advanced lock-free or reduced-lock implementation.

Do **not** start with lock-free structures. Correctness should come first.

---

7. Global queue

Tasks submitted by threads outside the scheduler need somewhere to go.

Therefore:

```
External Thread
      |
      v
Global Queue
      |
      v
Workers
```

A worker could follow this strategy:

```
1. Check local queue
2. Check global queue
3. Attempt work stealing
4. Sleep/wait
```

That creates a clear execution policy.

---

8. Priority scheduling

Support:

```C++
Priority::High
Priority::Normal
Priority::Low
```

Example:

```C++
scheduler.submit(
    Priority::High,
    [] {
        handle_urgent_job();
    }
);
```

The scheduler should prefer higher-priority tasks.

But you should also think about **starvation**.

For example:

```Plain text
1000 High tasks
10 Low tasks
```

If high-priority tasks keep arriving, low-priority tasks might never run.

Later you can add **priority aging**.

That is an excellent systems-design topic for interviews.

---

9. Task dependencies

This is where ThreadForge starts becoming more tha thread pool.

9. Task dependencies

This is where ThreadForge starts becoming more than a thread pool.

Imagine:

```
A ----\
       → C → D
B ----/
```

C should only execute once A and B complete.

A possible API could eventually look like:

```cpp
auto A = scheduler.submit(...);
auto B = scheduler.submit(...);

auto C = scheduler.submit_after({A, B}, ...);
```

Internally you could maintain:

```
dependency counter
```

For example:

```
Task C
remaining_dependencies = 2
```

After A:



```
Plain text
```

After A:

```
remaining_dependencies = 1
```

After B:

```
remaining_dependencies = 0
```

Now C enters the runnable queue.

This introduces interesting concepts:

```
task graphs
DAG scheduling
atomic counters
dependency resolution
```

---

10. Task lifecycle

Each task can conceptually have states:

```
Created
   ↓
WaitingForDependencies
   ↓
Ready
   ↓
Running
   ↓
Completed
```

```
Plain text
   ↓
Completed
```

Additional states:

```
Plain text
Cancelled
Failed
```

A possible enum:

```cpp
enum class TaskState {
    Created,
    Waiting,
    Ready,
    Running,
    Completed,
    Cancelled,
    Failed
};
```

---

11. Exception handling

Suppose a task throws:

```cpp
scheduler.submit([] {
    throw std::runtime_error("something failed");
});
```

The worker thread **must not crash**.

Instead:

```
task exception
      ↓
captured
      ↓
stored in std::future
      ↓
future.get()
      ↓
exception rethrown
```

Using:

```cpp
std::packaged_task
std::future
```

makes this manageable.

---

12. Graceful shutdown

Support at least:

```cpp
scheduler.shutdown();
```

The scheduler should:

```
stop accepting tasks
       ↓
finish queued tasks
```

```
        ↓
wake sleeping workers
        ↓
workers exit
        ↓
join threads
```

Also optionally support:

```cpp
scheduler.shutdown_now();
```

which cancels queued tasks where possible.

RAII should also ensure:

```cpp
~Scheduler()
```

does not leak threads.

---

13. Thread synchronization

You should intentionally demonstrate knowledge of:

```cpp
std::thread
std::jthread
std::mutex
std::condition_variable
std::atomic
std::future
std::promise
```

C++

```
std::condition_variable
std::atomic
std::future
std::promise
std::packaged_task
```

Prefer `std::jthread` where appropriate because C++20 provides cleaner lifetime management.

But don't use concurrency primitives merely to show them off. Every primitive should solve a real problem.

---

14. Metrics system

This is one of the most important parts for making the project impressive.

Track metrics such as:

Plain text

```
Tasks submitted
Tasks completed
Tasks stolen
Failed steal attempts
Queue wait time
Execution time
Scheduler overhead
Worker idle time
Worker utilization
Throughput
```

Then produce results such as:

Plain text

```
Workers:              8
Tasks:                1,000,000
Completed:            1,000,000
Execution time:       2.31 s
Throughput:           432,900 tasks/sec
```

```
Tasks stolen:           83,421
Average queue latency:  17.2 µs
CPU utilization:        94%
```

That makes the repository much more credible.

---

15. Benchmarking

Create several benchmarks.

Benchmark A — sequential

```
Single-thread execution
```

Benchmark B — naive thread pool

```
One shared queue
N workers
```

Benchmark C — ThreadForge work stealing

```
Local queues
+
global queue
+
work stealing
```

Compare:

```
Sequential
vs
Centralized Thread Pool
vs
ThreadForge
```

---

16. Benchmark workloads

Use multiple workload types.

CPU-bound

Examples:

```
matrix operations
prime number calculation
Fibonacci
hash computation
```

Tiny tasks

For testing scheduler overhead:

```
task execution = few microseconds
```

Uneven workloads

For example:

```
Worker 1 tasks: 100 ms
```

```
Worker 1 tasks: 100 ms
Worker 2 tasks: 1 ms
Worker 3 tasks: 10 ms
```

This is particularly useful for demonstrating why work stealing works.

Mixed workloads

```
short tasks
medium tasks
long tasks
```

---

17. Testing requirements

Do not build the project without serious tests.

Use:

```
GoogleTest
```

or Catch2.

Test:

```
basic task execution
task return values
thousands of tasks
concurrent submissions
exceptions
worker shutdown
task priorities
```

```
dependencies
work stealing
scheduler destruction
queue correctness
```

Also perform stress testing.

For example:

```
1,000,000 tasks
16 workers
multiple producer threads
```

---

18. Race-condition checking

On Linux, compile with ThreadSanitizer:

```
-fsanitize=thread
```

Also test with:

```
AddressSanitizer
UndefinedBehaviorSanitizer
```

This becomes valuable evidence in your README:

```
✓ ThreadSanitizer clean
```

```
✓ ThreadSanitizer clean
✓ AddressSanitizer clean
✓ 1M task stress test passed
```

---

19. Build system

Use:

```
CMake
```

Minimum:

```
CMake 3.20+
C++20
```

Commands should eventually look like:

```bash
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

---

20. Recommended folder structure

```
|   └── stress_test.cpp
|
```

Plain text

```
.
├── benchmarks/
│   ├── shared_queue_benchmark.cpp
│   ├── stealing_benchmark.cpp
│   ├── scaling_benchmark.cpp
│   └── workload_benchmark.cpp
│
├── examples/
│   ├── basic.cpp
│   ├── futures.cpp
│   ├── priorities.cpp
│   └── dependency_graph.cpp
│
├── scripts/
│   └── run_benchmarks.sh
│
└── docs/
    ├── architecture.md
    ├── scheduler.md
    └── benchmarks.md
```

---

21. Development phases

I strongly recommend having Codex build it incrementally rather than attempting all features simultaneously.

Phase 1

Build:

Plain text

```
CMake project
Scheduler
workers
global task queue
submit()
future support
shutdown
basic tests
```

Phase 2

Introduce:

```
per-worker queues
worker-local task submission
work stealing
steal counters
```

Phase 3

Add:

```
priorities
fairness
queue statistics
```

Phase 4

Add:

```
task dependencies
dependency graph
task state management
```

Phase 5

Add:

```
metrics
Google Benchmark
performance comparison
```

Phase 5

Add:

```
metrics
Google Benchmark
performance comparison
```

Phase 6

Add:

```
stress testing
sanitizers
CI
documentation
examples
```

Phase 7 — advanced

Potential extensions:

```
CPU affinity
NUMA awareness
lock-free queues
task cancellation
timeouts
coroutines
custom allocators
GPU task scheduling
```

Do **not** put all Phase 7 features into the first version.

22. Definition of success

The project is ready for your portfolio when you can confidently demonstrate:

```
✓ arbitrary task execution
✓ futures
✓ multiple workers
✓ local queues
✓ global queue
✓ work stealing
✓ safe shutdown
✓ priorities
✓ dependency scheduling
✓ exception propagation
✓ metrics
✓ benchmarks
✓ stress tests
✓ sanitizers
✓ documentation
```

More importantly, you should be able to explain:

> Why did you make this architectural decision?

That's more important in an interview than simply having lots of code.